

NominaAPP

Sistema de administración de construcción asistido por inteligencia artificial

Documento: Explicación técnica del proyecto en lenguaje llano

Propósito: Tesis de maestría

Empresa: THE HOUSE & CO LUCIANO SARKIS, S.R.L.

Año: 2026

Documento descriptivo de la arquitectura, herramientas, alojamiento y metodología de desarrollo asistido por inteligencia artificial.

Índice

1. ¿Dónde se escribe la aplicación?

2. ¿En qué idioma se le habla a la computadora?

3. ¿Cómo está organizada la aplicación por dentro?

4. ¿Dónde se guarda toda la información?

5. ¿Cómo se conecta la inteligencia artificial al desarrollo?

6. ¿Qué inteligencia artificial se usa exactamente?

7. ¿Dónde "vive" la aplicación una vez terminada?

8. ¿Cómo es el ciclo completo de trabajo?

9. ¿Cómo se prueba que la aplicación funcione bien?

10. ¿Por qué se eligió este conjunto de herramientas?

11. Resumen visual del sistema

12. Mini glosario para el jurado

13. Síntesis en una sola frase

1. ¿Dónde se escribe la aplicación?

La aplicación se escribe en un programa llamado **Cursor**. Cursor es como Microsoft Word, pero en vez de servir para escribir cartas o documentos, sirve para escribir las instrucciones (el código) que la computadora va a ejecutar.

Cursor está basado en otro programa muy popular llamado **Visual Studio Code**, pero con una diferencia importante: trae integrada la inteligencia artificial dentro del propio editor. Es decir, mientras el desarrollador escribe, puede pedirle a la IA que escriba o modifique partes del código por él, sin tener que salir del programa.

Cursor funciona en cualquier computadora moderna: Windows, Mac o Linux.

2. ¿En qué idioma se le habla a la computadora?

Las computadoras no entienden español ni inglés, entienden lenguajes de programación. La aplicación está escrita principalmente en un lenguaje llamado **TypeScript**.

Una analogía útil: si la computadora fuera un albañil, TypeScript sería como un cuaderno donde le escribimos paso a paso qué tiene que hacer y con qué materiales. La ventaja de TypeScript sobre otros lenguajes es que **revisa los errores antes de ejecutar el trabajo**, igual que un maestro de obra que revisa los planos antes de empezar a construir. Esto es muy importante en un sistema financiero donde un error en una suma puede costar dinero real.

Para que la aplicación se vea como una página web moderna, se usa una herramienta llamada **React**. React permite construir la pantalla por piezas reutilizables, como armar una casa con bloques de Lego: el bloque "tabla de nóminas", el bloque "formulario de contratistas", el bloque "menú lateral", etc. Cada bloque se programa una sola vez y se reutiliza en muchas pantallas.

Los colores, tamaños, espacios y diseño visual se manejan con **Tailwind CSS**, que es como una caja de herramientas de estilos predefinidos: en lugar de inventar cada color y cada margen, se usan clases ya listas (por ejemplo: "fondo blanco", "texto grande", "espacio mediano").

3. ¿Cómo está organizada la aplicación por dentro?

Por dentro la aplicación está dividida en **capas**, como una empresa con departamentos:

- **La pantalla** (lo que ve el usuario): solo muestra información y recibe clics. No piensa, no calcula, no guarda nada.
- **El intermediario**: cuando el usuario hace clic en un botón, este intermediario decide qué hay que hacer.
- **El servicio**: es el único que habla con la base de datos. Guarda, busca, modifica o elimina información.
- **La base de datos**: es donde queda guardada toda la información de la empresa (proyectos, contratistas, nóminas, etc.).

¿Por qué dividirlo así? Por la misma razón que en una constructora la persona que recibe al cliente no es la misma que mezcla el cemento. Cada parte hace lo suyo, y si mañana hay que cambiar el proveedor de cemento, no hay que cambiar también a la recepcionista.

Esto permite que la aplicación crezca sin volverse un desorden, y que un error en una parte no afecte a las demás.

4. ¿Dónde se guarda toda la información?

Todos los datos del sistema (proyectos, contratistas, nóminas, presupuestos, cubicaciones, fotos, firmas) se guardan en un servicio llamado **Supabase**.

Supabase es como **alquilar un espacio en un edificio profesional** en lugar de construir uno propio. En vez de comprar servidores, instalarlos, mantenerlos y protegerlos, simplemente se contrata el servicio y Supabase se encarga de todo:

- Guarda la información en una base de datos profesional llamada **PostgreSQL** (una de las más usadas y confiables del mundo).
- Protege quién puede ver qué (un usuario de una empresa nunca puede ver datos de otra empresa).
- Hace copias de seguridad automáticas todos los días.
- Permite que la información se actualice en tiempo real (si dos personas están viendo la misma nómina y una la modifica, la otra ve el cambio al instante).
- Guarda archivos como fotos de obra, firmas digitales y documentos.

La gran ventaja: **no hay servidores físicos que mantener ni administradores de sistemas que pagar**. Todo se gestiona desde una página web.

Para presentaciones o pruebas, la aplicación tiene un **modo demostración** que funciona sin conectarse a Supabase, usando datos de ejemplo guardados temporalmente en el navegador.

5. ¿Cómo se conecta la inteligencia artificial al desarrollo?

La inteligencia artificial se conecta a través del editor Cursor. El desarrollador escribe instrucciones en español, como si le hablara a un asistente, y la IA responde escribiendo o modificando el código.

Hay tres formas de pedirle ayuda a la IA:

5.1 Conversación (Chat)

El desarrollador escribe algo como: *"agrega un campo para registrar el porcentaje de retención del 5% en la tabla de cubicaciones"*. La IA lee el código relacionado, propone los cambios y se los muestra al desarrollador, que decide si acepta o rechaza cada modificación, línea por línea.

Es como tener un programador asistente al lado, al que se le explica qué se quiere y él lo redacta, pero el desarrollador siempre tiene la última palabra.

5.2 Sugerencias automáticas

Mientras el desarrollador escribe, la IA va adivinando qué quiere escribir a continuación y lo sugiere en gris. Si el desarrollador está de acuerdo, presiona la tecla Tab y la sugerencia se inserta. Es similar al teclado predictivo del celular, pero para código.

5.3 Modo Agente (autónomo)

Es la modalidad más avanzada. En vez de proponer cambios uno por uno, el desarrollador le da una tarea completa: *"crea un módulo de préstamos a contratistas con descuento automático por nómina"*. La IA trabaja sola: lee los archivos que necesita, escribe el código en varios sitios a la vez, ejecuta las pruebas, corrige errores y al final reporta lo que hizo.

5.4 Las "reglas de la casa" (.cursorrules)

En el proyecto hay un archivo especial llamado `.cursorrules` que funciona como un **manual de la empresa para la IA**. En ese archivo está escrito:

- Qué tecnologías se usan.
- Cómo se debe escribir el código (estilo, nombres, formato).
- Las reglas del negocio dominicano: el ITBIS es 18%, las cédulas tienen este formato, el descuento por cheque es 0.15%, etc.
- Los límites: ningún archivo puede tener más de 200 líneas, ninguna función más de 40 líneas.

Cada vez que el desarrollador le pide algo a la IA, Cursor le entrega también este manual. Así la IA respeta las reglas del proyecto y no inventa cosas que no encajan.

5.5 Conexión directa con la base de datos

Cursor tiene una funcionalidad llamada **MCP** (un puente que conecta la IA con servicios externos). Gracias a esto, la IA puede mirar directamente cómo está organizada la base de datos en Supabase, hacer consultas y proponer cambios al esquema, sin que el desarrollador tenga que copiarle y pegarle nada manualmente.

5.6 Agentes en la nube

Cursor permite delegar tareas a "trabajadores virtuales" que corren en computadoras remotas. El desarrollador les asigna una tarea (por ejemplo: *"agrega un módulo de control de calidad para ensayos de hormigón"*) y el trabajador virtual hace todo el trabajo solo: descarga el proyecto, escribe el código, prueba que funcione, y deja una propuesta lista para que el desarrollador la revise y la apruebe.

Esto permite avanzar en varias funcionalidades al mismo tiempo, como tener un equipo de programadores asistentes trabajando en paralelo.

6. ¿Qué inteligencia artificial se usa exactamente?

El proyecto usa principalmente la familia de modelos llamada **Claude**, desarrollados por una empresa llamada **Anthropic**. Hay dos versiones:

- **Claude Sonnet**: la versión equilibrada, rápida y suficientemente inteligente para la mayoría de las tareas. Se usa día a día.

- **Claude Opus:** la versión más potente, pero más lenta y costosa. Se reserva para tareas complicadas: diseñar arquitectura nueva, modificar la base de datos a gran escala, o resolver problemas que requieren analizar muchas partes del sistema a la vez.

La decisión de cuál modelo usar para cada tipo de tarea también está documentada en el archivo de reglas del proyecto.

7. ¿Dónde "vive" la aplicación una vez terminada?

La aplicación está dividida en dos partes que viven en sitios diferentes:

7.1 La parte visible (lo que el usuario ve en su navegador)

Vive en un servicio llamado **Vercel**. Vercel es como **una empresa de mensajería global**: cuando un usuario en Sosúa abre la aplicación, los archivos se le entregan desde un servidor cercano (Miami, por ejemplo), no desde el otro lado del mundo. Esto hace que la aplicación cargue muy rápido sin importar dónde esté el usuario.

Vercel también se encarga de:

- **Publicar automáticamente cada cambio:** cada vez que el desarrollador termina un cambio y lo guarda, Vercel lo publica solo, sin intervención manual.
- **Crear versiones de prueba:** cada propuesta de cambio genera una dirección web única donde se puede probar la nueva funcionalidad antes de publicarla oficialmente.
- **Dar seguridad** (HTTPS) y un dominio gratis.

Vercel tiene un plan gratuito suficiente para el tamaño de este proyecto académico.

7.2 La parte invisible (la base de datos y la información)

Vive en **Supabase**, que a su vez funciona sobre la infraestructura de **Amazon Web Services (AWS)**, la nube más grande del mundo. AWS es donde alojan sus servicios empresas como Netflix, Airbnb o Spotify, así que la confiabilidad es de nivel industrial.

7.3 El código fuente (las instrucciones originales)

Vive en **GitHub**, que es como un **Google Drive especializado para código**. GitHub guarda la historia completa del proyecto: quién cambió qué, cuándo y por qué. Si algo se rompe, se puede volver a una

versión anterior con un clic.

GitHub está conectado con Vercel: cada vez que se sube un cambio a GitHub, Vercel lo recibe automáticamente, lo prepara y lo publica.

8. ¿Cómo es el ciclo completo de trabajo?

Un día típico de desarrollo se ve así:

1. **Idea:** el desarrollador identifica algo que falta o que hay que mejorar.
2. **Consulta:** revisa el documento `PROJECT.md`, donde está escrito el estado actual del proyecto y qué módulos ya existen.
3. **Petición a la IA:** le explica a la IA en español qué quiere construir.
4. **Propuesta de la IA:** la IA escribe el código y muestra los cambios.
5. **Revisión:** el desarrollador acepta lo que está bien, corrige lo que no y rechaza lo que sobra.
6. **Pruebas:** se ejecutan automáticamente las pruebas que verifican que nada se rompió.
7. **Publicación:** el cambio se sube a GitHub, y Vercel lo publica solo.
8. **Documentación:** se actualiza el documento del proyecto con lo nuevo.

Todo este ciclo, que tradicionalmente podía tomar muchas horas, con asistencia de IA puede completarse en minutos para tareas sencillas y en menos tiempo del normal para tareas complejas.

9. ¿Cómo se prueba que la aplicación funcione bien?

El sistema tiene dos tipos de pruebas automáticas:

- **Pruebas de partes pequeñas (unitarias):** con una herramienta llamada **Vitest**. Verifican que las cuentas matemáticas estén bien (que el ITBIS calcule 18%, que la validación de cédula funcione, que las conversiones de moneda sean exactas). Hoy hay 51 pruebas de este tipo.
- **Pruebas de flujos completos:** con una herramienta llamada **Playwright**. Simulan a un usuario real abriendo el navegador, dándole clic a botones y completando procesos enteros: por ejemplo, "crear una nómina, aprobarla e imprimirla". Si en algún paso algo falla, la prueba avisa.

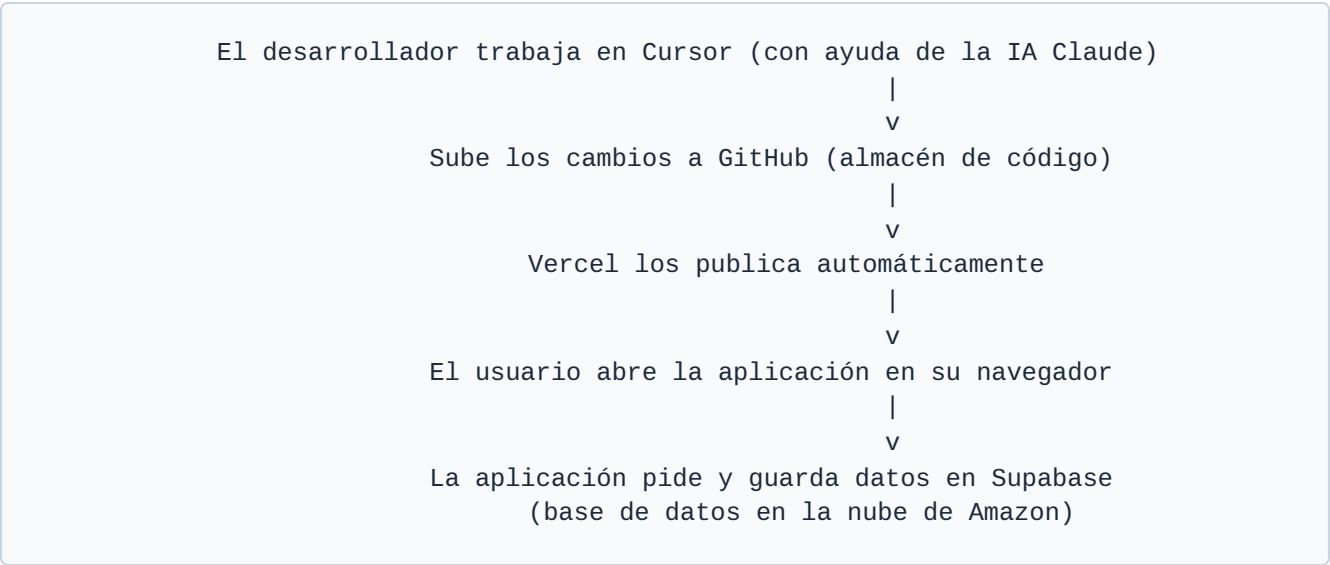
Estas pruebas se corren automáticamente cada vez que se hace un cambio, así nadie publica accidentalmente algo roto.

10. ¿Por qué se eligió este conjunto de herramientas?

La elección no fue arbitraria. Se basó en cuatro criterios:

Criterio	Razón
Costo	Todas las herramientas tienen versiones gratuitas suficientes para el tamaño del proyecto. No hay que pagar servidores, ni licencias, ni administradores.
Velocidad	Combinar React + Supabase + IA permite construir en semanas lo que antes tomaba meses con métodos tradicionales.
Estándares	Son las herramientas más usadas en la industria de software actualmente. Cualquier programador del mundo las conoce, lo que facilita contratar ayuda en el futuro.
Crecimiento	Si mañana el sistema tiene que atender 10 veces más usuarios, la infraestructura crece sola sin tener que reconstruir nada.

11. Resumen visual del sistema



12. Mini glosario para el jurado

Código

Las instrucciones escritas que la computadora ejecuta.

Editor

El programa donde se escribe el código (Cursor, en este caso).

Framework

Una caja de herramientas pre-fabricadas que evita tener que empezar de cero (React, en este caso).

Base de datos

El lugar donde se guarda la información de forma organizada.

Servidor

Una computadora siempre encendida que atiende peticiones de los usuarios.

Nube

Servidores que están en internet, alquilados a empresas como Amazon, Microsoft o Google.

IA o LLM

Inteligencia artificial capaz de entender y escribir texto (incluyendo código).

Repositorio

La carpeta donde vive todo el código del proyecto, con su historia completa.

Despliegue

El acto de publicar la aplicación para que el usuario la use.

Hosting

El servicio que aloja la aplicación en internet.

13. Síntesis en una sola frase

"La aplicación se escribe en un editor llamado Cursor que tiene inteligencia artificial integrada, se programa en un lenguaje llamado TypeScript usando React para las pantallas, guarda toda su información en una base de datos en la nube llamada Supabase, su código vive en GitHub y se publica automáticamente en Vercel para que cualquier usuario pueda usarla desde su navegador."